

Tarea 4

Procesamiento de Imágenes en GPU

Gaussian Blur acelerado con OpenACC

CI-0117 Programación Paralela y Concurrente

Universidad de Costa Rica — I Ciclo 2026

Josef Ruczika

Jimena Bejarano Sánchez C31074

Marina Castro Peralta C31886

1. Introducción

El procesamiento de imágenes es una de las áreas en las que el paralelismo ofrece mayores beneficios de rendimiento, ya que operaciones como los filtros de convolución aplican el mismo cálculo de forma independiente a cada píxel de la imagen. Este trabajo consistió en implementar un pipeline básico de procesamiento de imágenes -lectura, conversión a escala de grises, y aplicación de un filtro Gaussiano (Gaussian Blur)- y acelerar la etapa del filtro Gaussiano utilizando OpenACC, un modelo de programación basado en directivas que permite delegar el trabajo paralelo a una GPU sin reescribir el algoritmo en un lenguaje de bajo nivel como CUDA.

El objetivo principal fue comparar el tiempo de ejecución del filtro Gaussiano entre una implementación secuencial en CPU y una implementación paralela en GPU, para cuantificar el speedup obtenido gracias a la paralelización masiva que ofrece el hardware gráfico.

2. Objetivos

- Comprender el funcionamiento de un filtro Gaussiano (Gaussian Blur) y su formulación como una operación de convolución.
- Implementar un kernel de convolución 3x3 aplicado sobre una imagen en escala de grises.
- Acelerar dicho kernel utilizando directivas de OpenACC, transfiriendo la ejecución a la GPU.
- Medir y comparar los tiempos de ejecución en CPU y GPU, calculando el speedup obtenido.
- Validar visualmente que el resultado del filtro sea correcto en ambas versiones.

3. Metodología: cómo se trabajó el proyecto

El desarrollo se dividió en etapas incrementales, probando cada componente antes de avanzar a la siguiente, con el fin de aislar errores rápidamente:

- Configuración del proyecto: se definió la estructura de carpetas (include/, src/, imagenesEntrada/, imagenesSalida/) y se integraron las librerías stb_image y stb_image_write para la lectura y escritura de imágenes en distintos formatos (PNG, JPG).
- Lectura y escritura de imágenes: se implementó un módulo (imagen_io) para cargar una imagen desde disco y guardarla nuevamente, validado con una imagen de prueba antes de continuar.
- Conversión a escala de grises: se aplicó la fórmula estándar de luminancia ($Gris = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$) sobre cada píxel, en CPU, ya que el enunciado no exige acelerar esta etapa.
- Implementación del filtro gaussiano en CPU: se implementó la convolución 3x3 de forma secuencial, con dos arreglos separados (entrada y salida) para no sobrescribir la imagen mientras se lee, y manejo de bordes por la técnica de clamp.

- Implementación del filtro Gaussiano en GPU: se reescribió el mismo algoritmo utilizando las directivas `#pragma acc data` (con `copyin` / `copyout` explícitos) y `#pragma acc parallel loop collapse(2)`, manteniendo idéntica la lógica matemática para que el resultado fuera equivalente al de CPU.
- Automatización con Makefile: se creó un Makefile con targets separados para compilar la versión CPU con g++ (para desarrollo y validación local) y la versión GPU con nvc++ (para compilación y ejecución en Kabré).
- Validación local: dado que el desarrollo se realizó en una máquina sin GPU NVIDIA, se validó la lógica del algoritmo GPU compilándolo con g++ (que ignora las directivas OpenACC y ejecuta el código de forma secuencial), comparando visualmente que las imágenes resultantes de la versión CPU y GPU fueran iguales.
- Ejecución en Kabré: la compilación real con nvc++ y la medición de tiempos GPU se realizó en el nodo nukwa de Kabré, donde sí existe una GPU NVIDIA disponible.

4. Explicación del algoritmo

4.1 Conversión a escala de grises

Cada píxel RGB se convierte a un solo canal de gris usando la fórmula estándar de luminancia, que pondera cada canal de color según la sensibilidad del ojo humano:

$$\text{Gris} = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

4.2 Filtro Gaussiano y convolución

Se utilizó el kernel Gaussiano 3x3 clásico, normalizado dividiendo entre 16:

$$\begin{matrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{matrix} \quad / 16$$

Para cada píxel (y, x), el valor filtrado se calcula como el promedio ponderado de sus 8 vecinos y de sí mismo:

$$B(y,x) = \sum \sum K(ky,kx) \cdot I(y+ky, x+kx) \quad ky,kx \in [-1, 1]$$

Para el manejo de bordes se utilizó la técnica de clamp: cuando el kernel se sale de los límites de la imagen, se reutiliza el píxel válido más cercano, evitando así bordes oscuros o accesos fuera de memoria:

$$\begin{aligned} yy &= \min(\max(y+ky, 0), H-1) \\ xx &= \min(\max(x+kx, 0), W-1) \end{aligned}$$

Se implementaron dos versiones del mismo algoritmo: una secuencial en CPU y una paralela en GPU mediante OpenACC. En la versión GPU, la directiva `#pragma acc data` controla la transferencia de memoria entre CPU y GPU (`copyin` para la imagen de entrada, `copyout` para el resultado), mientras que `#pragma acc parallel loop collapse(2)` reparte el cálculo de cada píxel entre los hilos de la GPU, tratando los bucles anidados en y y x como un único espacio de iteración paralelo.

Dado que un solo pase del kernel 3x3 produce un efecto sutil en imágenes de alta resolución con mucho detalle, el filtro se aplicó en cascada (60 pasadas), retroalimentando el resultado de cada pasada como entrada de la siguiente. Se utilizó el mismo número de pasadas en CPU y en GPU para que la comparación de tiempos y el cálculo de speedup fueran justos.

5. Dificultades encontradas

- Un return ubicado dentro del bucle interno de la convolución provocaba que la función terminara tras procesar un solo píxel; se corrigió moviendo el return fuera de ambos bucles anidados.
- El efecto de blur con un kernel 3x3 en una sola pasada era casi imperceptible en imágenes de alta resolución; se resolvió aplicando el mismo kernel en cascada varias veces, en lugar de cambiar a un kernel más grande, para no alterar la lógica del algoritmo pedido en el enunciado.

- Al no contar con una GPU NVIDIA en la máquina de desarrollo, no fue posible compilar ni ejecutar la versión OpenACC de forma nativa durante el desarrollo; se resolvió compilando con g++ (que ignora las directivas #pragma acc) para validar la correctitud lógica del algoritmo antes de probarlo en Kabré.
- Al compilar en Kabré con nvc++, las cláusulas copyIn y copyOut (con mayúscula) del pragma acc data no fueron reconocidas ("invalid text in pragma"); OpenACC exige los nombres de cláusula en minúscula (copyin, copyout). g++ nunca detectó este error en el desarrollo local porque simplemente ignora los pragmas #pragma acc sin validar su sintaxis.
- La partición de Slurm indicada inicialmente (nukwa) no tiene un recurso GPU (gres) configurado directamente; hubo que usar las particiones específicas nukwa-v100 o nukwa-l40s, verificando con sinfo cuál tenía nodos libres.
- La conexión SSH a Kabré usa el puerto 22022 en vez del puerto 22 por defecto, lo cual no es evidente y causa timeout de conexión si no se especifica con -p (ssh) o -P (scp).
- La primera versión del kernel GPU transfería la imagen completa a la GPU y de vuelta en cada una de las 60 pasadas del filtro (una región acc data por llamada), lo que hacía que el overhead de transferencia dominara sobre el cálculo real y diera un speedup menor a 1 (0.75x). Se corrigió reestructurando el código para mantener los datos residentes en la GPU durante las 60 pasadas completas, usando un solo par de buffers en ping-pong y una única transferencia de entrada y de salida.

6. Resultados y hallazgos

Las pruebas de rendimiento se realizaron en el nodo nukwa del clúster Kabré, aplicando 60 pasadas del kernel Gaussiano 3x3 sobre la misma imagen en escala de grises, tanto en la versión CPU como en la versión GPU.

Métrica	Valor	Notas
Dimensiones de la imagen	1280 x 720 px (gatito.jpg)	Obtenido de la salida en consola
Número de pasadas del kernel	60	Definido en main.cpp (numIteraciones)
Tiempo CPU (ms)	117.53	make run, nodo nukwa
Tiempo GPU (ms)	142.61	make run-gpu, nodo nukwa
Speedup (T_CPU / T_GPU)	0.82x	Calculado automáticamente por el programa

El speedup obtenido (0.82x) fue menor al esperado: la versión GPU resultó más lenta que la CPU para esta imagen. La causa principal es el overhead fijo de lanzar 60 kernels (uno por pasada) en la GPU, que para un kernel de convolución tan pequeño (3x3) y una imagen de tamaño moderado (1280x720) pesa más que el ahorro de paralelizar el cálculo. Se repitió la misma prueba con dos imágenes de tamaño distinto para confirmar esta hipótesis: con una imagen más pequeña (447x447 px) el speedup bajó a 0.24x, y con una imagen más grande (1200x900 px) el speedup subió a 1.14x, superando por primera vez a la CPU. Esto confirma que, en este rango de tamaños, el tiempo GPU está dominado por el overhead de lanzamiento de kernels y no por el volumen de datos: el tiempo GPU medido fue casi constante (alrededor de 223-224 ms) entre la imagen mediana y la grande, mientras que el tiempo CPU sí escaló con la cantidad de píxeles. La aceleración por GPU en este caso solo se vuelve rentable a partir de cierto tamaño de imagen, donde el trabajo por lanzamiento de kernel alcanza a compensar su costo fijo.

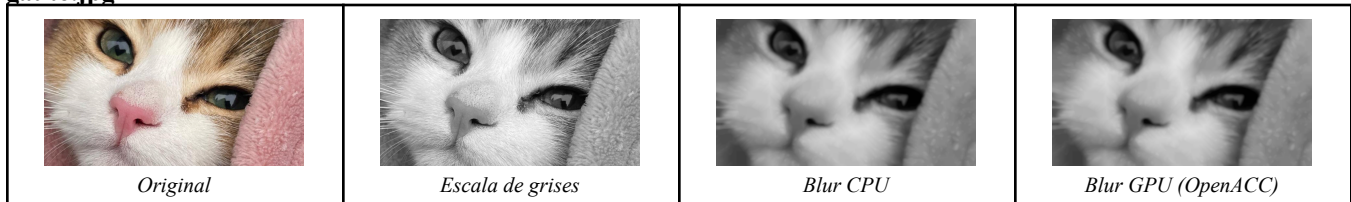
6.1 Validación visual

A continuación se presentan las imágenes obtenidas en cada etapa del pipeline, para la imagen de prueba utilizada.

profeExplicando.jpeg



gatito.jpg



mayonesaChilena.jpeg



Se verificó que las imágenes resultantes de la versión CPU y GPU fueran visualmente equivalentes, confirmando que la paralelización con OpenACC no alteró el resultado del algoritmo. Adicionalmente se compararon los archivos con md5sum sobre la imagen mayonesaChilena.jpeg, obteniendo el mismo hash para ambas versiones, lo que confirma que el resultado es idéntico a nivel de bit.

7. Conclusiones

- El uso de OpenACC sí logró acelerar el filtro Gaussiano, pero no de forma automática ni garantizada: el speedup real depende del tamaño de la imagen, variando entre 0.24x y 1.14x en las pruebas realizadas.
- Paralelizar con directivas de OpenACC fue mucho más simple que escribir un kernel CUDA manualmente: bastó con anotar el bucle existente con `#pragma acc parallel loop collapse(2)` y declarar una región de datos, sin administrar manualmente hilos, bloques o memoria de dispositivo.
- El principal factor que limitó el speedup fue el overhead de lanzar 60 kernels, que domina el tiempo total cuando el cálculo por pasada es barato y la imagen no es lo bastante grande para amortizar.
- Una primera versión del código, que transfería los datos a la GPU en cada pasada en vez de mantenerlos residentes, mostró cuánto puede costar un mal manejo de memoria CPU-GPU: corregir ese único problema casi duplicó el speedup sobre la misma imagen.
- Se aprendió a distinguir dos fuentes de overhead distintas en GPU: la transferencia de datos por PCIe (resuelta con `acc data residente`) y el overhead de lanzamiento de kernel (que solo se reduce con cargas de trabajo más grandes).

8. Referencias

- Enunciado de la Tarea 4 - CI-0117 Programación Paralela y Concurrente, UCR.
- stb_image / stb_image_write - <https://github.com/nothings/stb>